Embedding-Based Approaches for AI-Generated Text Detection

Mariam Gafar

Supervisor: Dr. Aya Salama

Faculty of Computer Engineering

German University in Cairo, Egypt

May 2025

©Mariam Gafar, 2025

Acknowledgements

Abstract

The rapid evolution of large language models (LLMs) such as ChatGPT has seen a spike in AI-generated text in many areas, sparking major concerns over authenticity, authorship, and academic integrity. As AI-generated content increasingly becomes indistinguishable from human writing, the need for effective detection methods has grown urgent. This thesis investigated various sentence embedding methodologies for identifying AI-written text and human-written text. The primary research question was whether combining static word embeddings (e.g., Word2Vec, GloVe) with contextual embeddings (e.g., BERT, T5) would provide better performance than utilizing standalone embeddings. To answer this question, various experiments were conducted using both standalone and hybrid embeddings and using SVM and XGBoost classifiers. Additionally, transformer-based LLMs were tested with different architectures (BERT encoder-only, GPT-Neo decoder-only, and T5 encoder-decoder) to analyze what effect different architectures would have on detection accuracy as well. The findings indicated that hybrid integrations of static and contextual embeddings exhibited superior performance compared to static and contextual embeddings when utilized independently. The T5 + Word2Vec + SVM model achieved the highest accuracy (97.17%) and F1-score (97.16%). On the other hand, the incorporation of GloVe into other models did not result in any advantageous outcomes. Furthermore, GPT-Neo as a decoder-only LLM was the weakest among other contextual models. In conclusion, combining contextual and static embeddings, especially Word2Vec, proved effective for AI-text detection. Future work can be done to explore fine-tuning contextual models, using other fusion techniques, and applying the system to real-world situations for general validation.

Abbreviations

NLP	Natural Language Processing
LLMs	Large Language Models
BERT	Bidirectional Encoder Representations from Transformers
T5	Text-To-Text Transformer
GPT-Neo	Generative Pre-trained Transformer
CNN	Convolutional Neural Network
SVM	Support Vector Machine
XGBoost	Extreme Gradient Boosting
LR	Logistic Regression
RF	Random Forest

Table of Contents

Acknowledgements i

Abbreviations **iii**

Abbreviations iii

List of Figures viii

List of Tables ix

1 Introduction **1**

1.1 Motivation 1

1.2 Problem Statement 3

1.3 Objectives 3

1.4 Thesis Outline 4

2 Background **5**

2.1 Concepts Overview 5

2.1.1 Natural Language Processing 5

2.1.2 Text Embeddings and Their Role in NLP 6

2.2 Contextual Embeddings: Transformer-Based Models 7

2.2.1 BERT: Bidirectional Encoder Representations from Transformers 7

2.2.2 T5: Text-To-Text Transfer Transformer 9

2.2.3 GPT-Neo: Generative Pre-trained Transformer 10

2.3 Static Embeddings 11

2.3.1 Glove: Global Vectors for Word Representation 11

2.3.2 Word2Vec: Predictive-Based Word Embeddings 12
2.4 Sentence Embedding and Classification Techniques 13
2.4.1 Convolutional Neural Networks (CNNs) 13
2.4.2 Traditional Classifiers: SVM and XGBoost 14
2.5 Discussion 16
2.5.1 Literature Review 16
2.5.2 Challenges in AI-Generated Text Detection 18

3 Methodology 20
3.1 Methodology Overview 20
3.2 Dataset 21
3.3 Data Pre-Processing 22
3.4 Software Setup 23
3.5 Tokenization and Embedding Extraction 24
3.5.1 Embeddings from Large Language Models (LLMs) 24
3.5.2 BERT Embeddings with Global Average Pooling 25
3.5.3 Static Embeddings with CNN and Global Max Pooling 26
3.5.4 Embedding Fusion 27
3.6 Model Training and Evaluation 27
3.7 Evaluation Metrics 28

4 Results and Discussion 31
4.1 Introduction 31
4.2 Results of Investigated Models 31
4.2.1 Performance of Single Embeddings 31
4.2.2 Performance of Combined Embeddings 35
4.2.3 Classifier Comparison: SVM vs XGBoost 37
4.2.4 Summary of Findings 38
4.3 Comparative Evaluation with Existing Methods 40

4.3.1 Re-Implemented Baseline Models 40
4.3.2 Analysis 40

5 Conclusion, Recommendations, and Future Work 42

5.1 Conclusion 42
5.2 Recommendations And Future Work 43

List of Figures

2.1 BERT's pre-training tasks: Masked Language Modeling and Next Sentence Prediction [18]. 8
2.2 BERT tokenization process using WordPiece tokenizer, splitting words into subwords and adding special tokens [21]. 9
2.3 T5 framework treats all NLP tasks as text-to-text by using task-specific prompts and generating textual outputs accordingly [23]. 10
2.4 Overview of the Word2Vec model architecture, showing the CBOW and Skip-Gram variants [13]. 12
2.5 Typical CNN architecture used for sentence-level feature extraction in NLP [13]. . 14

3.1 The proposed approach begins by preprocessing the input text, after which it generates sentence embeddings using two methods: T5 (a transformer-based model) and CNN-based embeddings derived from Word2Vec. These two sets of embeddings are then concatenated to form a single feature vector. Finally, this combined embedding is fed into an SVM (Support Vector Machine) classifier, which determines whether the input text is AI-generated or written by a human. 20
3.2 Python function used to clean and preprocess input text data. 23
3.3 Python code to extract sentence-level BERT embeddings using batch processing and mean pooling. 25
3.4 Python function to convert a sentence into a matrix of Word2Vec embeddings. . . . 26
3.5 CNN model architecture used to extract sentence-level embeddings from Word2Vec matrices. 27

3.6 Python code used to train the SVM classifier. 28
3.7 Python code used to train the XGBoost classifier. 28
3.8 Python code used to compute accuracy, precision, recall, and F1-score for both classifiers. 30

4.1 Confusion matrices for different embeddings with SVM classifier. Top: Contextual embeddings (BERT, T5); Bottom: Static embeddings (GloVe, Word2Vec). 34
4.2 ROC curve comparing T5 + Word2Vec + SVM with GloVe + SVM. 37
4.3 Classification accuracy comparison of different embedding models using SVM and XGBoost classifiers to show the performance impact of each approach. 38

List of Tables

2.1 Comparison of AI-Generated Text Detection Models 19

3.1 List of AI-Generated Text Datasets 21

3.2 Sample entries from the dataset used in this study. 22

4.1 Performance of BERT embeddings with SVM and XGBoost 32

4.2 Performance of GloVe embeddings with SVM and XGBoost 32

4.3 Performance of Word2Vec embeddings with SVM and XGBoost 32

4.4 Performance of GPT-Neo and T5 embeddings 33

4.5 Performance of BERT + GloVe embeddings 35

4.6 Performance of BERT + Word2Vec embeddings 35

4.7 Performance of T5 + GloVe and T5 + Word2Vec 36

4.8 Performance of re-implemented baseline models evaluated on our dataset. 40

Chapter 1

Introduction

1.1 Motivation

The latest advances in large language models (LLMs) and the increasing popularity of ChatGPT-like applications have blurred the boundary of high-quality text generation between humans and machines [1] ,this emergence has developed LLM-generated texts that are highly developed and almost similar to texts written by humans [2]. However, overdependence on these tools poses a great ethical challenge [3]. Their ability to generate coherent and context-appropriate text opens an opportunity for abuse, such as the spreading fake news and misinformation. These threats undermine the public's trust as well as distort society's perceptions. LLMs can also serve as tools of malicious manipulation over web content, inculcating perspectives into public opinion and political discourse [4]. As a result, one of the trending topics today in AI is the detection and identification of AI-generated text [5]. AI-generated text detection means employing AI technologies to identify and detect text content that is produced by an AI system and that may contain false and misleading information, or any content violating regulations [6].

Most generated texts are fundamentally based on embedding principles in the initial stages of any large language model (LLM), where a dense vector in space is extracted for each word to encapsulate the contextual information present in the training dataset [7]. In order to assist the several stages of text synthesis, the model uses an embedding that maps words or tokens into a high-

dimensional continuous space [8]. This embedding, coupled with additional layers and algorithms used in LLMs [9], informs the model's predictions for subsequent words. Understanding this generative pattern enables the identification of AI-suggested content versus natural human-like predictions, which are not strictly bound by patterns [8].

NLP and machine learning methods for AI-generated text detection have been developed to overcome these issues. These methods analyze the linguistic, statistical, or behavioral aspects of text to find abnormalities or deviations from human-generated material [10].

Deep learning plays a crucial role in AI generated text. Deep learning models learn linguistic patterns and regularities through training on large amounts of data and are able to generate texts with realism and coherence. Among them, pre-trained language models based on the Transformer architecture (e.g., GPT-3, BERT, etc.)are outstanding in natural language processing tasks, providing powerful support for AI-generated text [5]. Deep learning algorithms can capture hidden patterns and features in text by building complex neural network structures. Therefore, the application of embedding techniques, such as those derived from deep learning models such as BERT, improves the performance of detection systems by providing contextual insights that are vital for accurate classification [6].

To address the issues that arise from the efficient performance of deep learning techniques, many researchers are working on developing various methods to detect and identify problematic content in AI-generated texts. This includes rule-based, statistical methods, and machine learning techniques (e.g. support vector machines, random forests, etc.) to construct detection models and incorporate deep learning methods to improve detection accuracy [5].In this paper, a tool is developed for detecting LLM AI text generation by merging BERT with Word2vec and glove embeddings and sending the outcomes through a traditional classifier, with the expectation of improving the accuracy of AI text generation detection and providing some basis for future research.

1.2 Problem Statement

The use of AI-generated text, which is becoming increasingly sophisticated, poses a significant challenge in preserving trust and authenticity in online content. Current methods to detect AI-generated text often do not detect it very well or at all, especially as the models become more advanced and mimic the human writing style better. According to [4], while AI-generated text can always be detected, it may require many samples of text data. This problem is further aggravated by the potential for malicious use of AI-generated content, including the spread of misinformation and manipulation of public opinion.

Therefore, there is a need for a better and a more accurate way of detection of AI-generated text. This paper aims to address this problem by exploring the combination of contextualized embeddings from BERT together with the static embeddings of Word2Vec and GloVe , and then used as input features for traditional machine learning classifiers. By combining the strengths of both static and contextual embeddings, the aim is to provide better accuracy and reliability of detecting AI-generated text and, therefore, provide a powerful tool in combating the recent challenges of such increasingly sophisticated AI-generated text.

1.3 Objectives

The objectives of this paper are:

- **Investigate the Effectiveness of Individual Embeddings**
  - Compare the performance of BERT, Word2Vec, and GloVe embeddings individually in detecting AI-generated text.
- **Evaluate the Impact of Combining Contextual and Static Embeddings**
  - Assess how combining contextual embeddings from BERT with static embeddings from GloVe and Word2Vec affects AI-generated text detection.
- **Compare the Performance of Traditional Classifiers**

- Analyze the classification performance of Support Vector Machines (SVMs) and XGBoost when applied to individual and combined embeddings.

1.4 Thesis Outline

In this thesis, we explore embedding-based approaches for AI-generated text detection. In Chapter 1, we discuss the motivation behind the project and the significance of detecting AI-generated text. In Chapter 2, we provide an in-depth explanation of the embedding techniques used in this research, specifically focusing on BERT, GloVe, and Word2Vec. Chapter 3 describes the methodology, including how embeddings are combined and evaluated using traditional classifiers. Chapter 4 presents the experimental results and analysis, while Chapter 5 concludes with findings and future research directions.

- **Concept Overview:** A brief explanation of all technologies used throughout the paper, including embeddings and classification techniques.
- **Literature Review:** A compilation of all previous work done in the past regarding the detection of AI-generated text and the usage of different embedding-based approaches.
- **Methodology:** Workflow overview on how the embeddings were combined and evaluated using SVM and XGBoost.
- **Results:** A comparative analysis of various embedding strategies—both standalone and hybrid—as well as the performance of different classifiers used in the study.
- **Conclusion:** A brief discussion of key findings and possible future work that could be done to improve AI-generated text detection.

Chapter 2

Background

2.1 Concepts Overview

2.1.1 Natural Language Processing

Natural Language Processing (NLP) is a field within artificial intelligence. It focuses on enabling computers to comprehend, interpret, and create human language. It also provides a link between human communication and the understanding of computers, by changing unstructured text into a form that computers can recognize [11]. For instance, NLP is crucial in areas such as machine translation , sentiment analysis, and importantly, the detection of AI-generated text. Academic integrity, combating false information, and trusting material seen online are among the consequences resulting from the difficulty to identify AI-generated writing [1]. Thus, there is an important role for NLP capabilities and methodologies to help address the gap.

Key contributions of NLP to AI text detection include:

- **Feature Extraction:** NLP has the ability to examine and extract distinguishable features such as syntactic structures, semantic relationships, and properties of the text and its statistics [12]. These features then serve as the basis for classification model training.

- **Classification:** NLP provides methods for training machine learning models to effectively differentiate between human writing and AI-generated writing. These machine learning models can include both traditional models as well as deep learning models [5][6].

- **Contextual Understanding:** Using recent state-of-the-art NLP models to create contextualized embeddings allows models to identify words in context. These embeddings can provide meaning and concept in ways that non-contextual embeddings would not.

2.1.2 Text Embeddings and Their Role in NLP

Text embeddings are an important part of modern natural language processing (NLP). They act as an important intermediate measure between the human-readable text that we interact with and the numerical representation that computer processes. This means that we can transform words, phrases, and even entire documents into vectors to encapsulate the relationships of meaning and contexts. Vector representations are the basis of all types of NLP applications, ranging from simple sentiment analysis tasks to more complex machine translation tasks. The underlying idea is to accurately map words to vectors that are near each other if they convey similar meanings or contexts.

Different embedding approaches provide varying levels of complexity. For example, older methods like Word2Vec [13] or GloVe take a static approach to embeddings in that each word is assigned a vector irrespective of context. While traditional methods did a good job of capturing the general relationship of meaning, these methods struggle with polysemous words (words with multiple meanings) as they could not make a distinguishing vector representation for a word that has multiple meanings.

More sophisticated models, like BERT and ELMo, generate contextualized embeddings. When producing a word's vector, these techniques account for the surrounding words. This is a significant step towards a deeper meaning of understanding language [14]. For example, the vector representation for the word "bank" will differ as a financial institution versus a river bank. As a result, contextualized embeddings are useful in contexts that require deeper understanding of

meaning, such as in AI text detection where small distinctions in language use may indicate machine generation.

2.2 Contextual Embeddings: Transformer-Based Models

Transformer-based models have revolutionized natural language processing and other fields, becoming a cornerstone of modern AI. These models rely on the transformer architecture, which uses self-attention mechanisms to understand context and relationships within data [15].

The key innovation of the transformer architecture is the attention mechanism. Instead of processing data sequentially like recurrent neural networks, transformers process entire sequences in parallel. The attention mechanism allows the model to weigh the importance of different parts of the input when processing each element. This enables the model to capture long-range dependencies more effectively than previous architectures [15].

Several variations of the transformer architecture have emerged, each tailored for specific tasks:

- **Encoder-Only Models:** Models like BERT primarily use the encoder part of the transformer. BERT is pre-trained to generate contextualized word embeddings that take into account the surrounding words [6].
- **Decoder-Only Models:** Models like GPT and its variants use only the decoder part of the transformer. These models are particularly well-suited for text generation tasks [16].
- **Encoder-Decoder Models:** Models like T5 use both the encoder and decoder. T5 reformulates all NLP tasks into a text-to-text format, allowing it to handle a wide range of tasks with a single model [17].

2.2.1 BERT: Bidirectional Encoder Representations from Transformers

BERT, a transformer-based model ,has made a major impact in the area of NLP since it can understand the meaning of text using a bidirectional approach . In contrast to standard sequential models that read the text from left to right or right to left, BERT analyzes the text from both

directions at the same time, resulting in a better understanding of context of text. One of the features of BERT's design that stands out is its pre-training phase which consists of two tasks: Masked Language Modeling (MLM), and Next Sentence Prediction (NSP). In MLM, some words are randomly masked, and the model learns to predict the masked words from the context it has; thus enabling the BERT model to learn the relationship of a word with its surroundings. The NSP task is designed to train the model to determine whether one sentence follows another in the source document. This helps BERT understand the relationships between sentences and capture higher-level context [6]. Therefore, it excels at handling distant dependencies, collecting context-aware embeddings, and adjusting to different NLP applications [10].

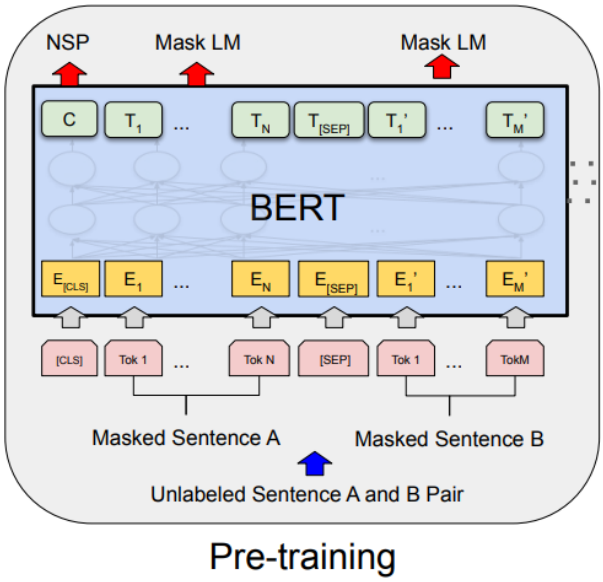


Figure 2.1: BERT's pre-training tasks: Masked Language Modeling and Next Sentence Prediction [18].

BERT processes input text using a technique called tokenization, which breaks down text into smaller units known as tokens. Tokens are the "building blocks" for BERT's language understanding and can be full words, subwords, or single characters. BERT only has a fixed list of tokens, or vocabulary, that it acquired in pre-training [19]. When it encounters a word that is not in this vocabulary, BERT attempts to break it down into smaller, known subwords using a method known

as WordPiece tokenization. This allows BERT to handle any out-of-vocabulary words while still retaining some of their semantic content.

Once the text has been tokenized, each token is transformed into a vector representation called an embedding. The embeddings, however, are not static; they were learned during BERT's pre-training as part of masked language modeling and next sentence prediction. As a result, BERT outputs high-dimensional embeddings that are context-dependent and capture a fair amount of semantic meaning about each token [6]. The embeddings serve as a basis for the different downstream natural language processing tasks [20].

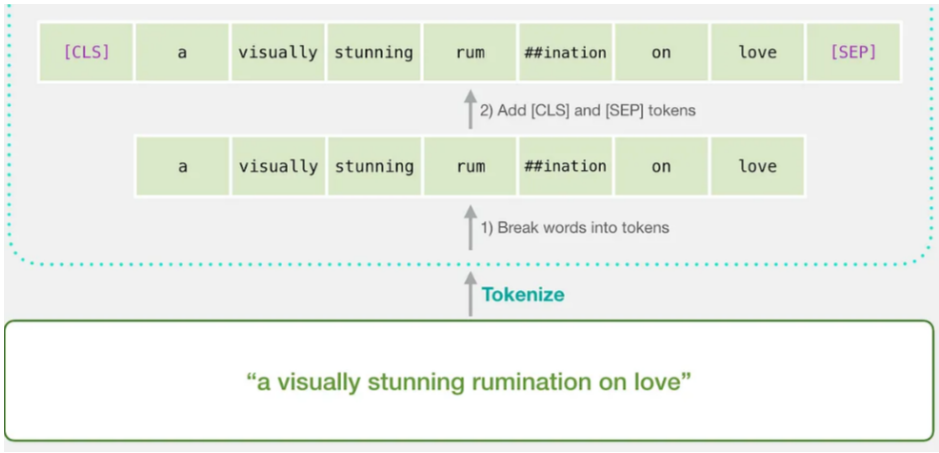


Figure 2.2: BERT tokenization process using WordPiece tokenizer, splitting words into subwords and adding special tokens [21].

2.2.2 T5: Text-To-Text Transfer Transformer

T5 is a language model that translates all NLP tasks to a text-to-text basis [17]. The premise of text-to-text means that T5 takes all text as input and it converts that text back into text regardless of the NLP task that's performed such as translation, summarization, answering questions and more. The T5 family consists of multiple sized models, allowing for flexibility in computational resources and performance [22]. This makes T5 a valuable tool in many natural language processing tasks.

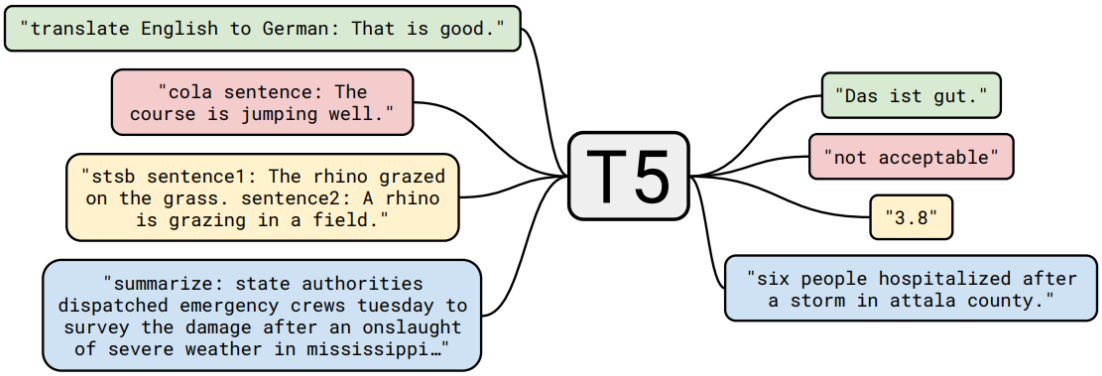


Figure 2.3: T5 framework treats all NLP tasks as text-to-text by using task-specific prompts and generating textual outputs accordingly [23].

T5 is an encoder-decoder model like the original Transformer model, where the encoder processes the input text, and the decoder generates the output text. This allows T5 to capture long-range dependencies and contextual information. Moreover, T5 is trained on the Colossal Clean Crawled Corpus (C4) which is a big dataset of text data scraped from the internet and this pre-training enables T5 to realize patterns and knowledge related to language, which can then be fine-tuned for specific tasks. As a result, T5 can be used perform tasks such as chatbots, machine translation, text summarization, and in our case AI-text detection. The text-to-text framework allows it to be adapted to different applications with very little change.

As a transformer-model, T5 uses embeddings to represent words (or moreprecisely, tokens) as continuous vectors. Generally, the embeddings for each token are learned during pre-training on the C4 dataset, where they learn some semantic information and relationships between words. Which makes it suitable to differentiate patterns or stylistic anomalies of AI-generated text [1].

2.2.3 GPT-Neo: Generative Pre-trained Transformer

GPT-Neo is an open-source transformer-based language model developed by EleutherAI, and is offered in a variety of parameter sizes: 125 million, 1.3 billion, and 2.7 billion [24]. It was designed as an open-source model alternative to models like GPT-3 by OpenAI. The goal was to create a transparent and accessible model that would attain comparable language generation

capabilities [25]. GPT-Neo is a language model based on the transformer architecture, which is a specific type of deep neural network architecture commonly used for NLP applications. Similar to other transformer-based language models, GPT-Neo uses self-attention mechanisms to weigh different words of a sentence to figure out their relative importance, upon which it learns context and is capable of generating text that is meaningful [16]. GPT-Neo is a decoder-only model. In other words, it is mainly designed for generating text where it forecasts the upcoming word in a sequence based on the previous words.

Like other transformer-based language models, GPT-Neo utilizes embeddings to represent words as continuous high-dimensional vectors. These embeddings are learned during the model's pre-training phase on large-scale corpora such as The Pile [24]. The embedding layer maps input tokens to vector representations that capture rich semantic and syntactic information. Unlike static word embeddings such as GloVe and Word2Ve which assign a fixed vector to each word regardless of context, GPT-Neo generates contextualized embeddings, where the representation of a word varies depending on its surrounding context. This dynamic nature allows the embeddings to better capture the nuances of language. Given these properties, embeddings from models like GPT-Neo can be leveraged in AI-generated text detection by feeding them into downstream classifiers, similarly to approaches using contextual embeddings from models like BERT or T5.

2.3 Static Embeddings

2.3.1 Glove: Global Vectors for Word Representation

GloVe, which stands for Global Vectors for Word Representation, is a widely used way to generate word embeddings. Unlike some methods that focus on local context, GloVe leverages the global statistics of word co-occurrences within a corpus to learn word representations [13].

GloVe's training procedure starts with the formation of a co-occurrence matrix where each entry represents the frequency of two words appearing in the same context. The model then learns word vectors in such a manner that the dot-product of two word vectors is close to the logarithm

of their co-occurrence frequency. This allows GloVe to capture semantic similarities among words based on their global co-occurrence patterns [13].

Some of the most significant features of GloVe are that it is able to capture global context and express semantic relationships in the vector space [13]. Therefore, GloVe embeddings can be utilized for a variety of NLP tasks, such as measuring word similarity, addressing word analogy problems, and supplying feature data for text categorization models [26]. However, GloVe produces static word embeddings, i.e., every word has a single vector representation in all contexts, which can be a drawback in some cases [14].

2.3.2 Word2Vec: Predictive-Based Word Embeddings

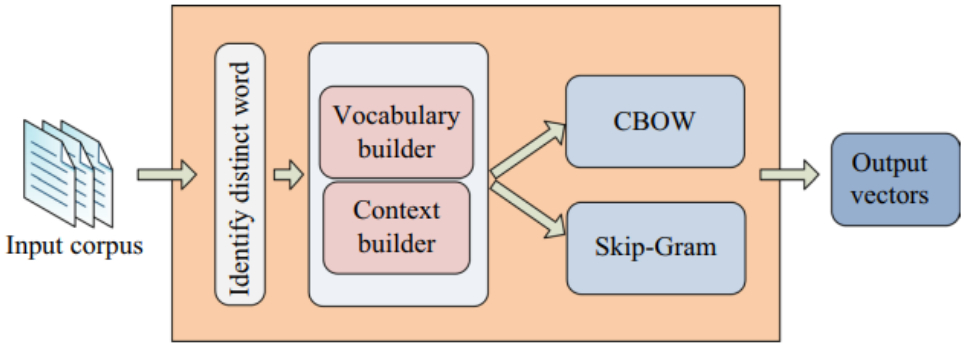


Figure 2.4: Overview of the Word2Vec model architecture, showing the CBOW and Skip-Gram variants [13].

Word2Vec is a frequently used method for creating word embeddings that was introduced by Tomas Mikolov and his colleagues at Google [27]. Its purpose is to capture the semantic meaning between words in a corpus [13]. The basic premise behind Word2Vec is that words that frequently appear in similar contexts are likely to also have similar vector representations [27].

Word2Vec has two main different architectures, Continuous Bag-of-Words and Skip-Gram. The Continuous Bag of Words (CBOW) architecture predicts a desired target word based on its surrounding context words. The Skip-Gram architecture does the opposite, it is designed to predict the context words given a target word. Both architectures employ a shallow neural network to

develop the corresponding word embeddings. This neural network can be trained on a vast text corpus, and the resultant weights are the word vectors [13].

Word2Vec marked a significant advancement in NLP by enabling the representation of words as continuous vectors, thereby capturing nuanced semantic relationships that earlier techniques could not achieve [28]. A fascinating characteristic of Word2Vec is the model's ability to perform semantic capturing using vector arithmetic. For instance, a well-known illustration is that the calculation vector("king") - vector("man") + vector("woman") yields a vector that is similar to vector("queen") [27].

2.4 Sentence Embedding and Classification Techniques

2.4.1 Convolutional Neural Networks (CNNs)

Convolutional neural networks (CNNs) are a type of deep learning model designed to carry out image recognition tasks. Because of their effectiveness in automatically and efficiently extracting local patterns, CNNs have become very powerful in Natural Language Processing (NLP) applications as well. When applied to text data, CNNs take in text as one- or two-dimensional signal, allowing them to detect local syntactic and semantic patterns through convolutional filters [29].

In the context of sentence embeddings, CNNs may be employed to generate fixed-size feature vectors from word embedding sequences, such as those generated by GloVe or Word2Vec. Such convolutional layers behave as n-gram detectors, which pick out significant phrase-level features at different window sizes. The features are then aggregated—typically via max-pooling—to create a compact representation for the entire sentence.

CNNs are particularly helpful when it comes to the carrying out of tasks such as AI-generated text detection because subtle linguistic differences between human-written texts and the texts that are generated by machine may well exist at the local phrase level. CNN-based architectures are effective in the majority of cases. According to recent studies, they can effectively detect AI-generated content. These studies do, in fact, show such capability. For example, Soni et al. introduced *TextConvoNet*, a CNN-based model that captures intra-sentence features as well as inter-

sentence features, as it convolves at many scales, which greatly improves classification accuracy on generated text datasets [30].

Moreover, Wang et al. proposed *SeqXGPT*, which leverages a series of convolutional layers with self-attention so that fine-grained features unique to AI-generated sentences are identified [31]. A prominent highlight of these diverse findings is that CNNs can readily produce meaningful sentence embeddings and are impressively competitive models in detecting text generated by large language models.

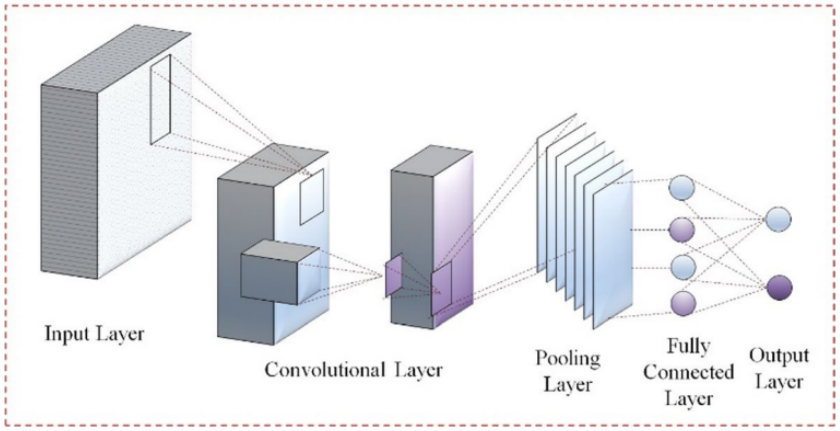


Figure 2.5: Typical CNN architecture used for sentence-level feature extraction in NLP [13].

2.4.2 Traditional Classifiers: SVM and XGBoost

Traditional machine learning algorithms offer robust solutions for AI text detection, particularly Support Vector Machines and XGBoost. While deep learning has gained popularity, these methods are still relevant and offer competitive performance due to their lower computational overhead.

Support Vector Machines (SVMs): SVMs are skillful in finding the optimal boundaries which separates the different classes of the item. For instance, in AI text classification, an SVM attempts to classify the text as human-written or AI-generated. An SVM is given the dataset of both categories and then creates a hyperplane in a polar feature space with the optimal distance between the two classes [32]. Instead of needing to create an entirely new classifier, SVMs can use kernels to learn complex decision boundaries allowing them to separate more difficult classes of

data that are not linearly separable. Most text classification tasks can be performed by SVM and therefore it is possible to use it for AI text detection [33]. [10] found that in applying SVMs for AI text detection, the accuracy rate was 0.81, thus further showing the effectiveness of SVMs.

XGBoost (XGB): XGBoost is a speedy and efficient gradient boosting algorithm. It consists of a combination of many decision trees, where each tree corrects the errors of its predecessors. This iterative process forms the backbone of a strong predictive model capable of capturing complex relationships in the data. In addition, XGBoost incorporates regularization techniques including L1 and L2, addressing over-fitting and increasing generalizability to unseen data . The algorithm is able to account for missing values and utilizes parallel processing to successfully work on larger datasets. XGBoost has been widely used to build classifiers [34] and there is an accuracy of 0.84 for AI text detection mentioned [10]. This speaks to the ability of XGBoost to be competitive within the AI text detection classification process.

2.5 Discussion

2.5.1 Literature Review

AI-Generated Text

Identifying AI-produced content is becoming even more difficult with the recent innovations brought forth by Large Language Models [1]. This has serious consequences for challenges such as identifying fake content, stopping AI-related plagiarism, and preventing wrongful accusations of using AI writing tools [1]. The more adopters of LLM in various uses, such as story telling, spell checking, and code completion, the greater the need for effective retargeting [35]. The ability of deep learning tools, especially transformers, to process, understand, and generate natural language brings with it serious consequences such as the dissemination of false or hate content [5]. Hence, there is an ongoing effort to design effective software that can detect text generated by AI through a multitude of techniques from basic statistical analysis to deep neural network architectures [35].

Deep Learning Approaches

Deep learning is the principal driving force behind AI text detection with powerful techniques that leverage the capacity of models like recurrent neural networks and transformers to deal with sequential data efficiently. The models have been built from massive human-written and machine-written text during training, and consequently, the detectors learn fine patterns and statistical deviations separating human-written from machine-written text [36]. Such variation could be seen in differences in structure of sentence, word frequency, and semantic consistency. This becomes increasingly significant in enhancing the detection methods when the generated texts become more and more complicated using Large Language Models in deep learning. Because of the constant evolution in architectures and training methods, these detectors are likely to remain in tune with recent developments in LLMs to tackle the problem of separating AI-generated text [37]. Active research is going towards setting up methods that do not give in to distributions in LLMs, thus making the detectors handle texts coming from unseen or changing models.

Traditional Machine Learning Approaches

Before deep learning went mainstream, traditional machine learning methods were the backbone of many text classification tasks, including the early endeavor of AI text detection. Such methods, for instance, Support Vector Machines and Naive Bayes classifiers, conventionally used the likes of TF-IDF to numerically portray text [38]. Essentially, they base their work on spotting statistical patterns and features in text, which is then patterned into categories and in the context of AI text detection, between human-nature and machine-generated text. Although these methods can attain reasonable accuracy for some tasks, they tend to struggle with the complexities of natural language [39]. One of their commonly recognized shortcomings includes their inability to recognize subtle semantic differences such as negation, or other figures of speech that impact the meaning significantly [40].

Embedding-Based Approaches

Embedding-based methods are prominent in detecting AI-generated text [38]. They convert text into numerical vectors (embeddings) capturing semantic and stylistic traits. Comparing these embeddings to known human and AI-written text helps identify text that is written by AI [41]. Pre-trained Large Language Models are commonly used to create these embeddings, with the resulting vectors serving as input for classifiers [42]. The continuous advancement of LLMs requires ongoing adaptation and refinement of embedding techniques for robust detection [38].

Comparison of Different Embedding Models

Different embedding models provide different trade-offs [43]. Simpler models like Word2Vec and GloVe can learn simple semantic relationships but are lacking with out-of-vocabulary words and fine-grained contexts. Character-level embeddings, however are effective in handling rare words and morphological variations and are therefore perfect for linguistically diverse texts but lack more abstract semantic understanding [44]. Other embeddings such as BERT are good at recovering context-dependent meaning but with the price of higher computational cost [45]. Therefore, the choice of the embedding model depends on the specific task, dataset, and available re-

sources. Hybrid models that integrate multiple embedding approaches have been explored to leverage each method's strength and improve detection accuracy in general [46].

Accuracy and Results of Embedding-Based Approaches

The accuracy of embedding-based models varies for various models, classifiers, and datasets [36]. [10] achieved 0.93 accuracy by using a BERT model. However, other studies with techniques like TF-IDF and classifiers like logistic regression, random forest, or SVC have yielded variable results [38], even lower at times. [38] discusses stylometric features' limited effectiveness in distinguishing between AI-generated real news and AI-generated false news, and emphasizes a move towards identifying human-written text versus AI-written text. [47] showed that it is hard for humans to distinguish AI-written text from human-written text. This distinction emphasizes the importance of careful model selection and calibration for consistent detection. Even slight changes in training data can significantly impact model performance [36].

2.5.2 Challenges in AI-Generated Text Detection

The task of detecting AI-generated text presents a number of significant challenges. The rapid evolution of LLMs [36][42] requires detection methods to constantly adapt to keep pace with the latest advancements in generative models. Achieving generalizability across different LLMs and domains is important in this regard [37], because a model trained on one LLM or dataset may not generalize to others. This highlights the need for developing valid and reliable methods for detecting AI-generated text regardless of the specific LLM or topic. Furthermore, the possibility of adversarial attacks, where generated text is deliberately modified to evade detection [1], poses a serious threat to the reliability of detection systems. The ongoing arms race between text generation and detection highlights the need for continuous research and development of more robust and adaptable detection methods, a point emphasized by various researchers in the field [36]. Finally, the ethical implications of using AI text detectors must be analyzed closely, especially in sensitive situations such as education [41], where misclassification could have serious consequences. Ad-

dressing these challenges requires a multi-faceted approach, combining technical advancements with careful consideration of the societal impact of these technologies.

Table 2.1: Comparison of AI-Generated Text Detection Models

Model Configuration	Accuracy	Year	Dataset(s)	Source
BERT	97.71%	2024	Author-Created	[6]
BERT	93.00%	2024	Author-Created	[10]
BERT + Logistic Regression	92.10%	2024	GPT-3D dataset	[48]
BERT	89.10%	2024	TweepFake	[42]
XGBoost	84.00%	2024	Author-Created	[10]
CNN with GloVe	83.87%	2024	Research Abstracts Dataset (RAD), AI-GA	[49]
SVM	81.00%	2024	Author-Created	[10]
BERT + Logistic Regression	78.12%	2024	Author-Created	[50]
CNN with Word2Vec	67.37%	2024	Research Abstracts Dataset (RAD), AI-GA	[49]
XGBoost	56.20%	2024	SemEval 2024 Task 8	[51]
SVM	52.29%	2024	SemEval 2024 Task 8	[51]
Glove + Random Forest	51.61%	2024	Author-Created	[50]
Word2Vec + LSTM	47.61%	2024	Author-Created	[50]

Chapter 3

Methodology

3.1 Methodology Overview

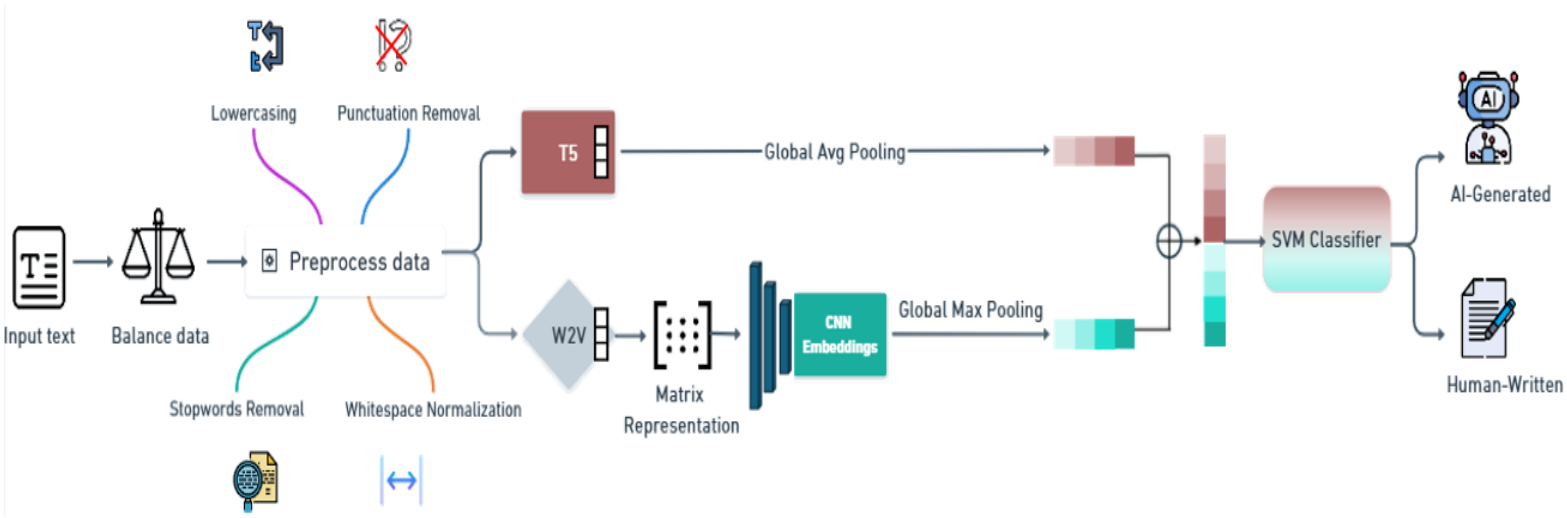


Figure 3.1: The proposed approach begins by preprocessing the input text, after which it generates sentence embeddings using two methods: T5 (a transformer-based model) and CNN-based embeddings derived from Word2Vec. These two sets of embeddings are then concatenated to form a single feature vector. Finally, this combined embedding is fed into an SVM (Support Vector Machine) classifier, which determines whether the input text is AI-generated or written by a human.

This research examines the use of embedding-based methods for detecting AI-generated text by comparing contextual embeddings derived from BERT with static embeddings from GloVe and Word2Vec, as well as assessing various combinations of these approaches. BERT embeddings are generated by averaging token-level representations, while GloVe and Word2Vec embeddings are converted into sentence-level features using a CNN. Next, these embeddings are fed into conventional classifiers, Support Vector Machines (SVM) and XGBoost, to test their performance. Not only that, but the combinations of BERT + GloVe, and BERT + Word2Vec were also tested to determine whether merging contextual and static embeddings enhances classification performance. A balanced dataset of human-written and AI-generated text was used for training and testing, as well as evaluation metrics: accuracy, precision, recall, and F1-score to measure the performance of the models.

3.2 Dataset

The dataset utilized in this research is the *"Augmented data for LLM - Detect AI Generated Text"*, which is publicly available on Kaggle [52]. This dataset is a curated and augmented collection derived from several existing contributions related to AI-generated and human-written text. Specifically, it compiles and cleans data from the following publicly shared datasets:

Table 3.1: List of AI-Generated Text Datasets

Dataset Name	**Source (Kaggle)**
LLM-Generated Essays	kaggle.com/radek1/llm-generated-essays
ArguGPT	kaggle.com/alejopaullier/argugpt
DAIGT (LLaMA 70B & Falcon180B)	kaggle.com/nbroad/daigt-data-llama-70b-and-falcon180b
DAIGT Proper Train	kaggle.com/thedrcat/daigt-proper-train-dataset
DAIGT v2 Train	kaggle.com/thedrcat/daigt-v2-train-dataset
Hello Claude (Anthropic Essays)	kaggle.com/darraghdog/hello-claude-1000-essays-from-anthropic
LLM 7 Prompt Training	kaggle.com/carlmcbrideellis/llm-7-prompt-training-dataset

For the purpose of this study, and to ensure a balanced and computationally manageable dataset, a subset of **6,000 samples** was randomly selected from the original dataset. This subset consists of **3,000 human-generated texts** labeled as `0` and **3,000 AI-generated texts** labeled as `1`. It was used for both training and evaluation of the proposed models. Overall, Table 3.2 provides an overview of what the dataset looks like.

Table 3.2: Sample entries from the dataset used in this study.

Text	Label
The Facial Action Coding System is an excellent way to...	0
Hey there! So, I guess I'm going to write about how working...	1
It is a great choice to have multiple people opinions...	0
...	...
The 21st century marks a period of unprecedented access to...	1
Driverless cars are important but will be more important in...	0
I believe that young people nowadays do not give enough time...	1

3.3 Data Pre-Processing

Effective data preprocessing is essential for the robust performance of machine learning models, particularly in tasks involving natural language processing. Preprocessing involves several steps to clean and prepare the text data for analysis. In this paper, the text is sequentially, converted to lowercase, removal of punctuation, stop words and empty spaces.

1. **Lowercasing:** All text was converted to lowercase to avoid the same word being regarded as different words due to different case, thus ensuring uniformity.

2. **Punctuation Removal:** All non-alphanumeric characters and punctuation marks were removed using regular expressions, to ensure that the model is trained strictly on textual elements.

3. **Stopwords Removal:** Common English stopwords (e.g., "the", "and", "is") were removed using NLTK's predefined list, as they contribute little semantic value to classification, thus minimizing data noise and enhancing focus on more distinctive text features.

4. **Whitespace Normalization:** Extra whitespaces were removed and replaced with single spaces, and leading / trail spaces were removed to unify the format and make the data more standardized.

This comprehensive preprocessing not only prepared the data set for effective training of the models, but also enhanced the accuracy of the models to distinguish between human and machine-generated texts, a critical aspect of the verification task.

The implementation of the preprocessing function is shown in the figure below 3.2. This code encapsulates the lowercasing, punctuation removal, stopword filtering, and whitespace normalization steps described above. It was applied to each sentence in the dataset, and the preprocessed output was saved to a CSV file for downstream use.

```
def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)
    text = ' '.join(word for word in text.split() if word not in stop_words)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

Figure 3.2: Python function used to clean and preprocess input text data.

3.4 Software Setup

The methods proposed in this research were executed using the Python programming language (version 3.10). For the preprocessing of text, the `pandas` library was employed to read and manipulate the dataset, while the `re` library was used to eliminate punctuation and unnecessary whitespace through regular expressions. Additionally, the nltk library facilitated the removal of English stopwords. To extract contextual embeddings, the `transformers` library from Hugging Face was utilized to load a pre-trained BERT model, and `torch` was utilized to run the model and extract sentence-level embeddings. In the case of static embeddings such as GloVe and Word2Vec, the `gensim` library was utilized. These static embeddings were then input into convolutional layers constructed with `tensorflow` and `keras`. For the classification task, the Support Vector

Machines (SVM) and XGBoost models were trained and evaluated using the `scikit-learn` and `xgboost` libraries, respectively. These libraries were chosen for their capability to support sophisticated models, thereby guaranteeing both reproducibility and scalability.

3.5 Tokenization and Embedding Extraction

In order to enable machine learning classifiers to process the textual input, each sentence was transformed into a fixed-length numerical vector representation, also known as an embedding. This study utilized both contextual embeddings (BERT, GPT-Neo and T5) and static embeddings (GloVe and Word2Vec) to capture semantic patterns from the data. These embeddings were extracted as sentence-level vectors, using model-specific tokenization and vectorization strategies.

3.5.1 Embeddings from Large Language Models (LLMs)

In addition to BERT and static embeddings, embedding extraction pipeline was extended to include sentence embeddings generated by large language models (LLMs), namely GPT-NEO and T5. The rationale behind this decision was to evaluate whether different transformer architectures—specifically decoder-only (GPT-NEO) and encoder-decoder (T5)—could offer improved performance over BERT (an encoder-only model).

For GPT-NEO, a decoder-only model designed primarily for language generation, the embeddings were extracted by passing the input sentence through the model and averaging the last decoder layer's hidden states across all tokens. The resulting fixed-size embedding vector was used directly for classification. Due to its lower performance in preliminary evaluations, GPT-NEO was used only as a standalone embedding source without static embedding fusion.

For T5, a sequence-to-sequence model, only the encoder was used for generating contextual embeddings. Sentences were tokenized using the T5 tokenizer, passed through the encoder, and the output hidden states were averaged across tokens to produce sentence-level embeddings. After observing that T5 embeddings outperformed BERT in isolation, additional combinations were tested by concatenating T5 embeddings with GloVe and Word2Vec representations, following the same

CNN-based procedure described earlier. These combined representations were then evaluated using traditional classifiers (SVM and XGBoost) to measure their effectiveness in distinguishing between human-written and AI-generated text.

3.5.2 BERT Embeddings with Global Average Pooling

For contextual embeddings, a pre-trained `bert-base-uncased` model was loaded from the Hugging Face `transformers` library, as well as its corresponding WordPiece tokenizer. Raw input sentences were tokenized using BERT's tokenizer, which divides the text into subword tokens and handles padding and truncation to a maximum sequence length of 512 tokens.

BERT outputs a contextualized vector for each token in the sequence. To derive a single sentence-level embedding from these token representations, global average pooling was applied over the final hidden layer (last hidden state) across all token positions. This operation produced one fixed-size embedding vector per sentence. To improve computational efficiency, the inputs were processed in batches of size 16. Finally, the extracted embeddings were stored and saved in a pickle file to be used later.

The following code illustrates how BERT sentence embeddings were extracted using batch processing and mean pooling over token embeddings:

```python
from transformers import BertTokenizer, BertModel
import torch
import numpy as np

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")
model.eval()

def get_bert_embeddings_batch(texts, batch_size=16):
    embeddings = []
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]
        tokens = tokenizer(batch.tolist(), padding="max_length",
        truncation=True, max_length=512, return_tensors="pt")
        with torch.no_grad():
            output = model(**tokens)
        batch_embeddings = output.last_hidden_state.mean(dim=1).numpy()
        embeddings.append(batch_embeddings)
    return np.vstack(embeddings)
```

Figure 3.3: Python code to extract sentence-level BERT embeddings using batch processing and mean pooling.

3.5.3 Static Embeddings with CNN and Global Max Pooling

In parallel, static embeddings were computed using pre-trained GloVe and Word2Vec word vectors. Unlike BERT, these models fixed the vector representation for each word in isolation without regard to the context in which they appear. During data preparation, every sentence was tokenized using simple whitespace splitting, and a matrix was constructed where each row contained the vector of a word in the sentence. Out-of-vocabulary words were represented as zero vectors, and shorter sentences were zero-padded to ensure uniform matrix dimensions.

The transformation of a sentence into a matrix of Word2Vec embeddings is illustrated in the code snippet below. This function tokenizes each sentence, retrieves the corresponding word vectors from the Word2Vec vocabulary, and constructs a fixed-size matrix by padding or truncating the input as necessary.

```
def sentence_to_matrix(text):
    words = text.split()[:MAX_LEN]
    matrix = np.zeros((MAX_LEN, EMBEDDING_DIM))
    for i, word in enumerate(words):
        if word in word2vec:
            matrix[i] = word2vec[word]
    return matrix
```

Figure 3.4: Python function to convert a sentence into a matrix of Word2Vec embeddings.

In order to get sentence-level representations useful for downstream classification and concatenation with BERT embeddings, a one-dimensional Convolutional Neural Network (CNN) was employed. The CNN architecture included a single `Conv1D` layer, succeeded by a GlobalMaxPooling1D layer and a dense projection layer. This design allowed the model to capture the most significant local features throughout the sequence. The resulting vectors served as sentence embeddings for classification purposes.

The architecture of the CNN used for embedding extraction is shown below. It takes a 2D sentence matrix as input and outputs a fixed-size sentence embedding.

```python
def build_cnn_model():
    model = keras.Sequential([
        layers.Input(shape=(MAX_LEN, EMBEDDING_DIM)),
        layers.Conv1D(filters=128, kernel_size=3, activation="relu"),
        layers.GlobalMaxPooling1D(),
        layers.Dense(512, activation="relu")
    ])
    return model
```

Figure 3.5: CNN model architecture used to extract sentence-level embeddings from Word2Vec matrices.

3.5.4 Embedding Fusion

To analyze the effect of combining contextual and static semantic information, the final BERT embeddings were concatenated with the corresponding GloVe or Word2Vec CNN-based embeddings. Two sets of hybrid representations were created: one combining BERT with GloVe and the other combining BERT with Word2Vec. Subsequently, these combined embeddings, along with each individual type, were then passed to traditional classifiers (SVM and XGBoost) to evaluate their effectiveness in distinguishing between human-written and AI-generated text.

3.6 Model Training and Evaluation

Following the concatenation of sentence embeddings, a series of experiments were conducted using traditional classifiers to evaluate the effectiveness of different embedding types in detecting AI-generated text. Specifically, Support Vector Machines (SVM) and eXtreme Gradient Boosting (XGBoost) were used to train classification models on the extracted embeddings.

The SVM classifier was implemented with the `scikit-learn` library with a linear kernel, suitable for high-dimensional vector spaces. XGBoost models were trained with the `xgboost` library, using default hyperparameters, owing to its efficiency and regularization ability in handling complex patterns.

To provide a clearer understanding of the model implementation, the figures below illustrate the code used to train both classifiers:

```python
from sklearn.svm import SVC

svm_model = SVC(kernel='linear', probability=True)
svm_model.fit(X_train, y_train)
```

Figure 3.6: Python code used to train the SVM classifier.

```python
from xgboost import XGBClassifier

xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
xgb_model.fit(X_train, y_train)
```

Figure 3.7: Python code used to train the XGBoost classifier.

All models were trained on a balanced dataset containing an equal number of AI-generated and human-written samples to avoid class imbalance. Each model was assessed with typical classification measurements, such as measuring accuracy, precision, recall, and F1 scores calculated on unseen test data.

This evaluation strategy allowed for the systematic comparison of contextual embeddings, static embeddings, and their combinations, helping to determine which representation is most useful in assisting traditional classifiers to distinguish between AI-generated text and human-written text.

3.7 Evaluation Metrics

To assess the performance of the classification models in distinguishing between AI-generated and human-written text, four standard evaluation metrics were used: accuracy, precision, recall, and F1-score. These metrics were calculated based on the True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN) values derived from the model's predictions [53].

- **Accuracy:** This metric measures the overall correctness of the model by calculating the ratio of correctly predicted instances (both true positives and true negatives) to the total number

of predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.1}$$

- **Precision:** Precision measures the proportion of positive predictions that were actually correct. In the context of AI-generated text detection, it reflects how many of the texts predicted as AI-generated truly are. High precision is important when false positives are costly.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.2}$$

- **Recall:** Also known as sensitivity, recall calculates the proportion of actual positive cases (e.g., AI-generated texts) that were correctly identified by the model. It is crucial in scenarios where missing true positives has a higher cost.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.3}$$

- **F1-Score:** The F1-score is the harmonic mean of precision and recall. It provides a balanced evaluation when both false positives and false negatives are important, making it especially suitable for this binary classification task.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.4}$$

These metrics were computed using the classification_report function from the `scikit-learn` library. They enabled a detailed comparison between the performance of different embedding strategies (BERT, GPT-Neo, T5 GloVe, Word2Vec, and combinations) and classifiers (SVM and XGBoost).

The following code snippet illustrates how these evaluation metrics were implemented in Python:

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

def evaluate_model(name, model, X_test, y_test):
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(f"Results for {name}:")
    print(f"Accuracy       : {acc:.4f}")
    print(f"Precision      : {prec:.4f}")
    print(f"Recall         : {rec:.4f}")
    print(f"F1 Score       : {f1:.4f}")

evaluate_model("SVM", svm_model, X_test, y_test)
evaluate_model("XGBoost", xgb_model, X_test, y_test)
```

Figure 3.8: Python code used to compute accuracy, precision, recall, and F1-score for both classifiers.

Furthermore, to complement the standard evaluation metrics, Receiver Operating Characteristic (ROC) curves and confusion matrices were also generated for each classifier. The ROC curve illustrates the trade-off between true positive rate (recall) and false positive rate at various classification thresholds, providing a comprehensive view of the model's ability to distinguish between classes. The Area Under the ROC Curve (AUC) was used as a summary statistic, where higher values indicate better separability between AI-generated and human-written text. Additionally, confusion matrices were employed to visualize the distribution of correct and incorrect predictions, helping to identify specific patterns of misclassification. These tools offer a deeper understanding of model behavior beyond aggregate scores, particularly useful when evaluating class-specific performance.

Chapter 4

Results and Discussion

4.1 Introduction

This chapter presents the experimental results obtained from training various models using different embedding techniques for AI-generated text detection. The models were evaluated using key classification metrics to compare their performance and understand the impact of using static versus contextual embeddings. Results from individual embeddings and combined embeddings are presented, followed by an interpretation of the findings.

4.2 Results of Investigated Models

4.2.1 Performance of Single Embeddings

BERT Embeddings

The BERT-based sentence embeddings were evaluated using both SVM and XGBoost classifiers. Table 4.1 summarizes the performance across both evaluation metrics.

Table 4.1: Performance of BERT embeddings with SVM and XGBoost

Model	Accuracy	Precision	Recall	F1-score
BERT + SVM	96.67%	96.98%	96.33%	96.66%
BERT + XGBoost	95.50%	95.65%	95.33%	95.49%

GloVe Embeddings

The static GloVe embeddings, processed through a CNN for sentence-level representation, were also evaluated using both classifiers.

Table 4.2: Performance of GloVe embeddings with SVM and XGBoost

Model	Accuracy	Precision	Recall	F1-score
GloVe + SVM	78.00%	78.97%	76.33%	77.63%
GloVe + XGBoost	78.58%	79.32%	77.33%	78.31%

Word2Vec Embeddings

Similarly, sentence embeddings derived from Word2Vec were tested.

Table 4.3: Performance of Word2Vec embeddings with SVM and XGBoost

Model	Accuracy	Precision	Recall	F1-score
Word2Vec + SVM	73.25%	74.35%	71.00%	72.63%
Word2Vec + XGBoost	75.08%	75.64%	74.00%	74.81%

The results clearly show that contextual embeddings from BERT significantly outperform static embeddings like GloVe and Word2Vec in AI-generated text detection. BERT-based models achieved the highest performance across all metrics, with SVM reaching 96.67% accuracy. This reinforces the strength of contextualized representations in capturing deeper semantic and syntactic patterns that are essential for distinguishing between human-written and machine-generated text.

In comparison, GloVe and Word2Vec, which assign fixed vectors to each word without considering context, showed notably weaker performance. Word2Vec, in particular, achieved only 73.25% accuracy with SVM, highlighting its limited capability for this task.

LLM Embeddings (GPT-Neo and T5)

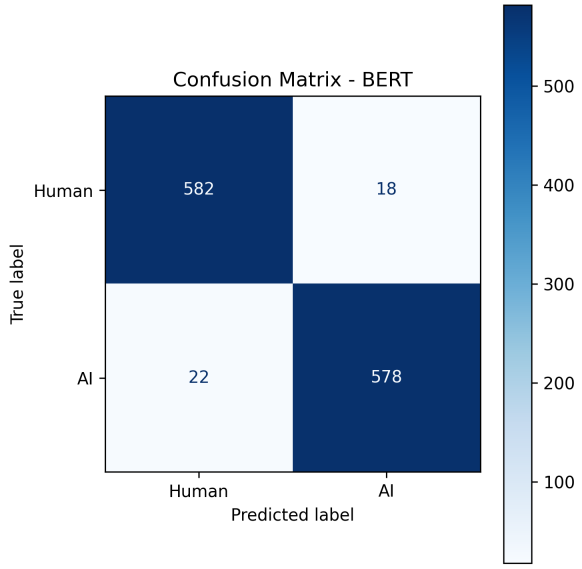
To expand the investigation, sentence embeddings were also extracted from large language models (LLMs): GPT-Neo (decoder-only) and T5 (encoder-decoder). These models were included to assess whether their architectural differences could provide better semantic understanding compared to BERT.

Table 4.4: Performance of GPT-Neo and T5 embeddings

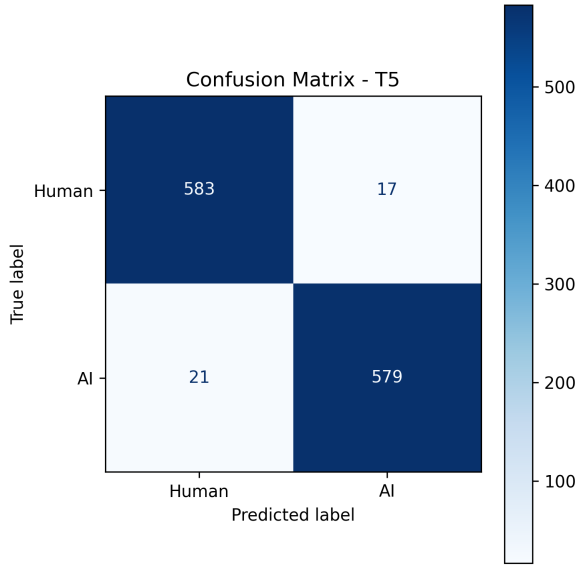
Model	Accuracy	Precision	Recall	F1-score
T5 + SVM	96.83%	97.15%	96.50%	96.82%
T5 + XGBoost	94.33%	94.93%	93.67%	94.30%
GPT-Neo + SVM	93.17%	93.02%	93.33%	93.18%
GPT-Neo + XGBoost	94.83%	96.86%	92.67%	94.72%

These results highlight that while GPT-Neo offers competitive performance, it lags behind BERT and T5 in accuracy and F1-score. T5, on the other hand, outperformed BERT when used alone achieving accuracy of 96.83%.

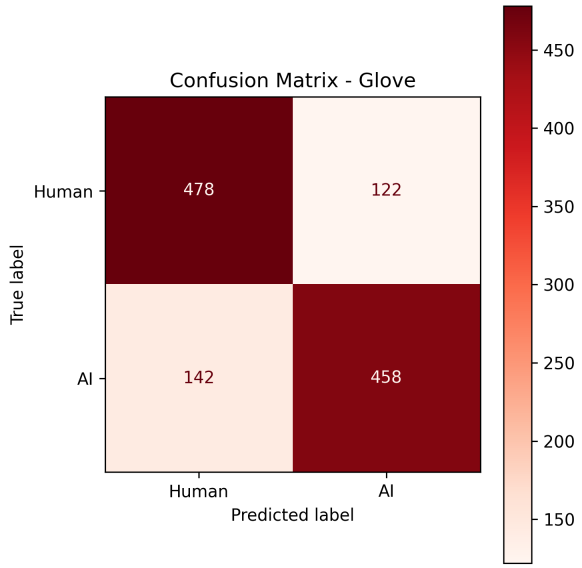
To further illustrate the difference in performance between contextual and static embeddings, Figure 4.1 presents the confusion matrices for four representative models. The top row shows the best-performing models based on contextual embeddings (BERT and T5), whereas the bottom row depicts the results of the lowest-performing models using static embeddings (GloVe and Word2Vec). This comparison visually reinforces how much better contextual models perform in distinguishing AI-generated text from human-written text, highlighting the significant performance gap between the two types of embeddings.



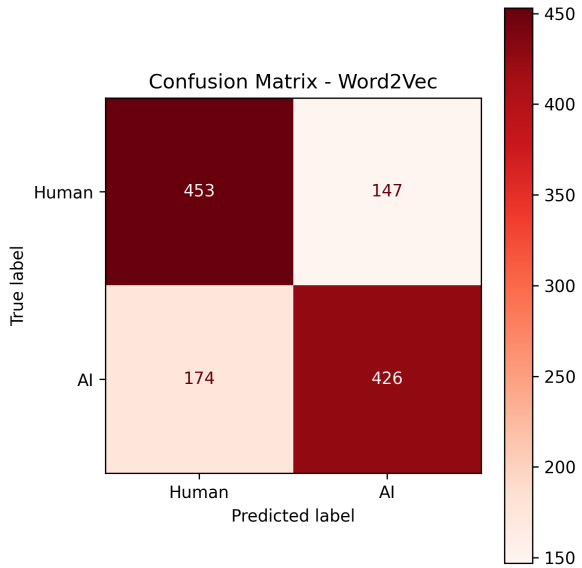
(a) BERT + SVM



(b) T5 + SVM



(c) GloVe + SVM



(d) Word2Vec + SVM

Figure 4.1: Confusion matrices for different embeddings with SVM classifier. Top: Contextual embeddings (BERT, T5); Bottom: Static embeddings (GloVe, Word2Vec).

4.2.2 Performance of Combined Embeddings

BERT + GloVe

The concatenation of BERT sentence embeddings with GloVe-based sentence vectors was evaluated to assess whether combining static and contextual embeddings improves performance.

Table 4.5: Performance of BERT + GloVe embeddings

Model	Accuracy	Precision	Recall	F1-score
BERT + GloVe + SVM	96.00%	95.85%	96.17%	96.01%
BERT + GloVe + XGBoost	95.67%	95.67%	95.67%	95.67%

BERT + Word2Vec

Similarly, BERT was combined with Word2Vec embeddings and evaluated using the same classifiers.

Table 4.6: Performance of BERT + Word2Vec embeddings

Model	Accuracy	Precision	Recall	F1-score
BERT + Word2Vec + SVM	96.75%	96.52%	97.00%	96.76%
BERT + Word2Vec + XGBoost	95.75%	96.29%	95.17%	95.73%

The combination of BERT with static embeddings led to modest but consistent improvements over using BERT alone. Notably, the BERT + Word2Vec model with SVM achieved the highest overall performance, reaching 96.75% accuracy and an F1-score of 96.76%. This suggests that Word2Vec may provide complementary local or syntactic features that enrich BERT's contextual understanding.

On the other hand, while the BERT + GloVe combination also improved over individual static embeddings, it did not outperform BERT alone, indicating that Word2Vec offers more useful synergy with BERT in this context.

T5 + GloVe and Word2Vec

Given the strong performance of T5 as a standalone embedding model, additional experiments were conducted by combining T5 with static embeddings (GloVe and Word2Vec), using the same CNN-based fusion approach applied in previous combinations.

Table 4.7: Performance of T5 + GloVe and T5 + Word2Vec

Model	Accuracy	Precision	Recall	F1-score
T5 + Word2Vec + SVM	97.17%	97.32%	97.00%	97.16%
T5 + Word2Vec + XGBoost	94.92%	95.14%	94.67%	94.90%
T5 + Glove + SVM	96.50%	97.13%	95.83%	96.48%
T5 + Glove + XGBoost	94.92%	95.76%	94.00%	94.87%

Just as predicted, Word2Vec not only improved preformance of BERT but it also improved T5 as well where the T5 + Word2Vec + SVM model achieved the highest overall performance, outperforming all previous configurations. This suggests that T5 embeddings capture rich contextual information, while Word2Vec complements them with local syntactic features.

To visually illustrate this improvement, Figure 4.2 presents the ROC curves comparing the best-performing model (T5 + Word2Vec + SVM) against the strongest purely static model (GloVe + SVM). The superior area under the curve (AUC) achieved by the fused model further reinforces the benefit of combining diverse embedding types.

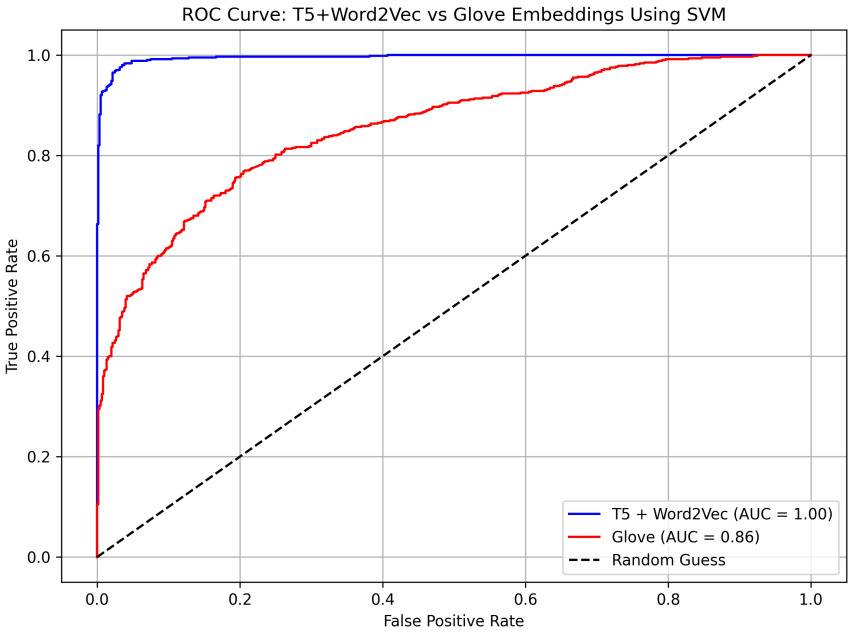


Figure 4.2: ROC curve comparing T5 + Word2Vec + SVM with GloVe + SVM.

4.2.3 Classifier Comparison: SVM vs XGBoost

Across all embedding configurations, SVM consistently outperformed XGBoost. This trend was observed for both single and combined embeddings. The superior performance of SVM may be attributed to its effectiveness in handling high-dimensional feature spaces and separating complex boundaries, which is particularly relevant when working with dense sentence embeddings.

To help visualize and compare the performance of the different embedding models used in this study, Figure 4.3 presents a bar chart summarizing their classification accuracies. Each embedding approach—BERT, T5, GPT-Neo, Word2Vec, GloVe, and their various combinations—was evaluated using both Support Vector Machine (SVM) and XGBoost classifiers. This comparison highlights the differences in performance across models and helps illustrate the impact of combining contextual and static embeddings on detection accuracy.

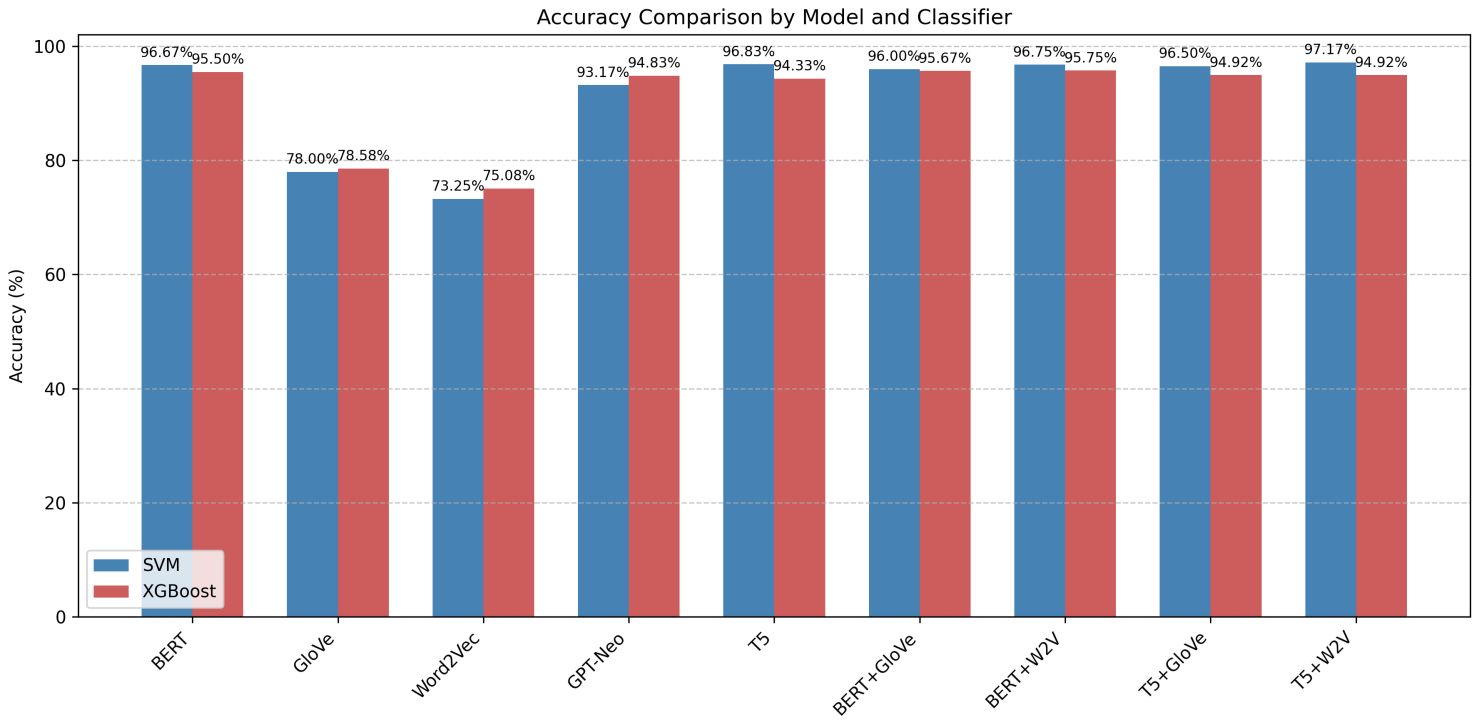


Figure 4.3: Classification accuracy comparison of different embedding models using SVM and XGBoost classifiers to show the performance impact of each approach.

4.2.4 Summary of Findings

The key outcomes from the experiments can be summarized as follows:

1. **Contextual embeddings outperform static ones:** Contextual embeddings from BERT, T5 and GPT-Neo consistently achieved better classification results than static embeddings from glove and word2vec. T5 was the best standalone model with accuracy of 96.83% while word2vec was the lowest accuracy of 73.25%. This indicates that contextual embeddings are more advantageous for capturing the rich semantic and syntactic relationships that distinguish human-written from AI-written text.

2. **Hybrid embeddings offer slight improvements:** Combining contextual embeddings from transformer models with static embeddings produced mixed results. GloVe did not yield observed gains in performance when combined with either BERT or T5, while Word2Vec

yielded slight improvements, with BERT and Word2Vec yielding a 0.08% increase in accuracy and a 0.65% increase in F1-score compared to BERT alone. The largest gain was seen with T5, achieving 0.34% gains in accuracy and F1-score when combined with word2vec. These results highlight the value of selectively integrating static embeddings like Word2Vec to boost the effectiveness of transformer-based models in AI-generated text detection.

3. **Different Transformer Architectures Yield Varying Results:** The performance of language models varied notably depending on their underlying architecture. T5, an encoder-decoder model, outperformed both BERT (encoder-only) and GPT-Neo (decoder-only) in detecting AI-generated text. The highest accuracy was achieved by combining T5 with static embeddings from Word2Vec, reaching 97.17% accuracy and 97.16% F1-score using SVM. These results suggest that the encoder-decoder architecture of T5 is particularly well-suited for this task, and its performance can be further enhanced by integrating static embeddings.

4. **GPT-Neo's limited performance:** GPT-Neo, a decoder-only model, was tested independently , even though it still performed better than static embeddings alone, it performed worse than both BERT and T5. Given this, no further experiments were conducted using GPT-Neo in combination with static embeddings, as its performance was unlikely to exceed that of T5 or BERT. Thus, GPT-Neo was excluded from combined embedding experiments.

5. **SVM outperform XGBoost in hybrid settings:** SVM turned out to be the best classifier when embeddings were combined, especially in high-dimensional representation. This implies that SVM is better able to exploit the richer feature spaces created by hybrid embeddings, which can be an advantage for integrated models.

6. **Implications for AI-generated text detection:** The results validate the importance of AI-generated text detection by transformer-based models like T5, which always gave the best results. Hybrid approaches, combining T5 with static embeddings like Word2Vec, gave the best results, hence indicating a strong benefit with hybrid models created from both contextual and static embeddings.

4.3 Comparative Evaluation with Existing Methods

To assess the effectiveness of the proposed approach, a comparative evaluation was conducted by re-implementing models from relevant literature and testing them on the dataset used in this research. This ensured a fair comparison, as all models were evaluated under identical conditions and data.

4.3.1 Re-Implemented Baseline Models

Models from three papers were selected:

- [10]: BERT + SVM
- [50]: BERT + Logistic Regression (LR), Word2Vec + LSTM, GloVe + Random Forest (RF)
- [48]: BERT + LR after applying PCA for embedding space restriction

The results of these models when tested on our dataset are shown in Table 4.8. For each model, we report accuracy, precision, recall, and F1 score.

Table 4.8: Performance of re-implemented baseline models evaluated on our dataset.

Model	Source	Accuracy	Precision	Recall	F1-score
T5 + Word2Vec + SVM	**Ours**	**97.17%**	**97.32%**	**97.00%**	**97.16%**
BERT + SVM	[10]	96.67%	96.98%	96.33%	96.66%
BERT + LR	[50]	95.08%	95.46%	94.67%	95.06%
Word2Vec + LSTM	[50]	86.83%	87.97%	85.33%	86.63%
GloVe + RF	[50]	92.75%	94.61%	90.67%	92.60%
BERT + LR (PCA)	[48]	69.17%	69.69%	67.83%	68.75%

4.3.2 Analysis

Our model achieves significantly higher performance across all metrics compared to the baseline models. Although BERT + SVM [10] reached an accuracy of 96.67%, as well as the best-

performing model from [50], BERT + LR, achieved an accuracy of 95.08%, our model T5 + Word2Vec + SVM still achieved the highest accuracy out of all models with an accuracy of 97.17%. This is a considerable improvement, especially in tasks involving a subtle semantic distinction, such as AI-generated text detection.

Several factors contribute to this improvement:

- **Hybrid Embedding Strategy:** Leveraging both contextual (T5) and static (Word2Vec) embeddings gives the model the opportunity to utilize both deep syntactic structure and surface-level semantic similarity. T5 learns fine language representations from pre-trained transformer architectures, while Word2Vec supplies distributional semantics learned from large-scale corpora and thus complements feature representations.

- **Effective Classifier Choice:** The Support Vector Machine classification methods work really well with our hybrid embeddings, which is not surprising because SVMs are quite effective in high-dimensional feature spaces. SVM methods are particularly effective when the number of features is much larger than the number of samples, which is often the case with text embeddings.

This comparative analysis confirms the superiority of our approach and supports our hypothesis that combining static and contextual embeddings provides a more robust representation, thereby enhancing accuracy in AI-generated text detection. Furthermore, by consistently outperforming re-implemented state-of-the-art models on the same dataset and under identical conditions, our method demonstrates a meaningful advancement in the field and establishes a new performance benchmark for this task.

Chapter 5

Conclusion, Recommendations, and Future Work

5.1 Conclusion

This research evaluated the effectiveness of various sentence embedding strategies for the task of detecting AI-generated text. According to the main hypothesis, classification performance would be improved by combining contextual and static embeddings rather than relying only on one type of embedding. Using two conventional classifiers, SVM and XGBoost, a systematic series of experiments was carried out to assess this, comparing models based on individual embeddings as well as hybrid combinations.

The results aligned with what was expected. Indeed, it was validated that static embeddings performed worse than contextual embeddings when used independently. The results aligned with what was expected. Indeed, it was validated that static embeddings performed worse than contextual embeddings when used independently. Among the individual models, BERT and T5 had the strongest performance while other static models like Word2Vec and GloVe were far behind. Moreover, combining contextual embeddings with static ones yielded significant improvements. In particular, combining Word2Vec with BERT and T5 achieved better results compared to using either BERT or T5 individually. The T5 + Word2Vec + SVM model achieved the best overall

performance, reaching an accuracy of 97.17% and an F1-score of 97.16%. This demonstrates that contextual representations will improve performance when concatenated with static embeddings such as Word2Vec which offer localized syntactic information. In contrast, GloVe embeddings contributed little when combined with contextual models, offering no clear advantage over using BERT or T5 alone. This indicates that not all static embeddings are useful for hybridization; therefore, importance must be placed on the selection of the embeddings to be combined.

To further investigate embedding diversity, the study also involved experiments with large language models (LLMs) of different transformer architectures. As BERT (encoder only) and T5 (encoder-decoder) were already included, GPT-Neo (decoder-only) was included to complete the comparison. Thus, the degree to which architectural differences between LLMs impacted detection performance could be assessed. While T5 slightly outperformed BERT when tested individually, GPT-Neo showed relatively poor performance and was removed from further hybrid experimentation. This suggested that even though standalone LLMs are useful for contextual understanding, the performance gains were achieved from hybrid embeddings and not LLM embeddings alone.

Overall, the findings of this research confirm that embedding strategy plays a very significant role in AI-generated text detection systems' performance. The prevalence of gains obtained with hybrid embeddings, particularly those combining contextual richness with Word2Vec's capture of linguistic patterns, speaks to the power of combining different semantic perspectives. The results indicate the usefulness of complementary data between embedding types and provide robust support for hybrid approaches to future text classification and AI- detection applications.

5.2 Recommendations And Future Work

This study highlights the effectiveness in combining static and contextual embeddings for detecting AI-generated content, yet there is plenty of room to build upon and deepen these findings. One natural next step is to go beyond frozen contextual embeddings by fine-tuning BERT, T5 or other similar models on the AI-detection task. This process of adapting the models to the domain

of detecting AI text could facilitate the capture of more subtle cues surrounding machine-generated text, particularly in complex real-world situations.

Moreover, additional methods with more complex feature fusion strategies should be explored. Instead of simple concatenation, subsequent work may focus on methods such as attention-based fusion, learnable weight schemes, or joint embedding projections. These methods may enable models to optimally exploit the static and contextual representations and respond to changes in the input dynamically.

Finally, another important direction for the future work is to expand the scope of evaluation. In this way, by applying the current framework to real-world, user-generated content, where human and AI texts are mostly interwoven in subtle ways, would help assess practical robustness and identify the boundaries of detection capability. Likewise, cross-lingual generalization and testing in specific domains (e.g., legal, medical, academic texts) would provide further evidence regarding the transferability and adaptability of this approach across different linguistic and thematic contexts.

In general, the findings of this study offer a solid basis for embedding-based hybrid detection. Future work on fine-tuning, smarter fusion, and larger corpora will further advance the development of accurate and robust AI-generated text detection systems.

Bibliography

[1] Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. Radar: Robust ai-text detection via adversarial learning. *Advances in neural information processing systems*, 36:15077–15095, 2023.

[2] Ruixiang Tang, Yu-Neng Chuang, and Xia Hu. The science of detecting llm-generated text. *Communications of the ACM*, 67(4):50–59, 2024.

[3] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[4] Souradip Chakraborty, Amrit Bedi, Sicheng Zhu, Bang An, Dinesh Manocha, and Furong Huang. Position: On the possibilities of ai-generated text detection. In *Forty-first International Conference on Machine Learning*, 2024.

[5] Yuhong Mo, Hao Qin, Yushan Dong, Ziyi Zhu, and Zhenglin Li. Large language model (llm) ai text generation detection based on transformer deep learning algorithm. *arXiv preprint arXiv:2405.06652*, 2024.

[6] Hao Wang, Jianwei Li, and Zhengyu Li. Ai-generated text detection and classification based on bert deep learning algorithm. *arXiv preprint arXiv:2405.16422*, 2024.

[7] Yoav Goldberg and Omer Levy. word2vec explained: deriving mikolov et al.'s negative-sampling word-embedding method. *arXiv preprint arXiv:1402.3722*, 2014.

[8] Ahmed K Kadhim, Lei Jiao, Rishad Shafik, and Ole-Christoffer Granmo. Adversarial attacks on ai-generated text detection models: A token probability-based approach using embeddings. *arXiv preprint arXiv:2501.18998*, 2025.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[10] Nuzhat Prova. Detecting ai generated text based on nlp and machine learning approaches. *arXiv preprint arXiv:2404.10032*, 2024.

[11] Nikolaos-Ioannis Galanis, Panagiotis Vafiadis, Kostas-Gkouram Mirzaev, and George A Papakostas. Machine learning meets natural language processing-the story so far. In *IFIP International Conference on Artificial Intelligence Applications and Innovations*, pages 673–686. Springer, 2021.

[12] Sainyam Galhotra, Behzad Golshan, and Wang-Chiew Tan. Adaptive rule discovery for labeling text data. In *Proceedings of the 2021 International conference on management of data*, pages 2217–2225, 2021.

[13] Deepak Suresh Asudani, Naresh Kumar Nagwani, and Pradeep Singh. Impact of word embedding models on text analytics in deep learning environment: a review. *Artificial intelligence review*, 56(9):10345–10425, 2023.

[14] Sawsan Alshattnawi, Amani Shatnawi, Anas MR AlSobeh, and Aws A Magableh. Beyond word-based model embeddings: Contextualized representations for enhanced social media spam detection. *Applied Sciences*, 14(6):2254, 2024.

[15] Matteo Boffa, Idilio Drago, Marco Mellia, Luca Vassio, Danilo Giordano, Rodolfo Valentim, and Zied Ben Houidi. Logprécis: Unleashing language models for automated malicious log analysis: Précis: A concise summary of essential points, statements, or facts. *Computers & Security*, 141:103805, 2024.

[16] Gokul Yenduri, Gautam Srivastava, Praveen Kumar Reddy Maddikunta, Rutvij H Jhaveri, Weizheng Wang, Athanasios V Vasilakos, Thippa Reddy Gadekallu, et al. Generative pre-trained transformer: A comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions. *arXiv preprint arXiv:2305.10435*, 2023.

[17] Naveen Ram, Dima Kuzmin, Ellie Ka In Chio, Moustafa Farid Alzantot, Santiago Ontañón, Ambarish Jash, and Judith Yue Li. Multi-task end-to-end training improves conversational recommendation. *arXiv preprint arXiv:2305.06218*, 2023.

[18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[19] Tiandeng Wu, Qijiong Liu, Yi Cao, Yao Huang, Xiao-Ming Wu, and Jiandong Ding. Continual graph convolutional network for text classification. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 13754–13762, 2023.

[20] Shah Khalid, Shah Khusro, Aftab Alam, and Abdul Wahid. Bert-embedding and citation network analysis based query expansion technique for scholarly search. *arXiv preprint arXiv:2301.11069*, 2023.

[21] Kamran Anwar. Illustration from: Word embedding methods: A review, 2021. Image used from article. Accessed: 2025-05-07.

[22] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[23] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[24] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

[25] Leon Fröhling and Arkaitz Zubiaga. Feature-based detection of automated language models: tackling gpt-2, gpt-3 and grover. *PeerJ Computer Science*, 7:e443, 2021.

[26] Ameeta Agrawal, Aijun An, and Manos Papagelis. Learning emotion-enriched word representations. In *Proceedings of the 27th international conference on computational linguistics*, pages 950–961, 2018.

[27] KENNETH WARD CHURCH. Word2vec. *Natural Language Engineering*, 23(1):155–162, 2017.

[28] Kian Kenyon-Dean, Edward Newell, and Jackie Chi Kit Cheung. Deconstructing word embedding algorithms. *arXiv preprint arXiv:2011.07013*, 2020.

[29] Yahui Chen. Convolutional neural network for sentence classification. Master's thesis, University of Waterloo, 2015.

[30] Sanskar Soni, Satyendra Singh Chouhan, and Santosh Singh Rathore. Textconvonet: A convolutional neural network based architecture for text classification. *Applied Intelligence*, 53(11):14249–14268, 2023.

[31] Pengyu Wang, Linyang Li, Ke Ren, Botian Jiang, Dong Zhang, and Xipeng Qiu. Seqxgpt: Sentence-level ai-generated text detection. *arXiv preprint arXiv:2310.08903*, 2023.

[32] Sohail R Reddy. A machine learning approach for modeling irregular regions with multiple owners in wind farm layout design. *Energy*, 220:119691, 2021.

[33] Dani Gunawan, Romi Fadillah Rahmat, Arsandi Putra, and Muhammad Fermi Pasha. Filtering spam text messages by using twitter-lda algorithm. In *2018 IEEE International Conference on Communication, Networks and Satellite (Comnetsat)*, pages 1–6. IEEE, 2018.

[34] Kouassi Simeon Kouassi, Mamadou Diarra, Kouassi Hilaire Edi, and Brou Jean-Claude Koua. Detection of cocoa leaf diseases using the cnn-based feature extractor and xgboost classifier. *Open Journal of Applied Sciences*, 14(10):2955–2972, 2024.

[35] Harika Abburi, Kalyani Roy, Michael Suesserman, Nirmala Pudota, Balaji Veeramani, Edward Bowen, and Sanmitra Bhattacharya. A simple yet efficient ensemble approach for ai-generated text detection. *arXiv preprint arXiv:2311.03084*, 2023.

[36] Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. Can ai-generated text be reliably detected? *arXiv preprint arXiv:2303.11156*, 2023.

[37] Lei Li, Yongfeng Zhang, and Li Chen. Prompt distillation for efficient llm-based recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 1348–1357, 2023.

[38] Tharindu Kumarage, Joshua Garland, Amrita Bhattacharjee, Kirill Trapeznikov, Scott Ruston, and Huan Liu. Stylometric detection of ai-generated text in twitter timelines. *arXiv preprint arXiv:2303.03697*, 2023.

[39] Tushar Abhishek, Daksh Rawat, Manish Gupta, and Vasudeva Varma. Transformer models for text coherence assessment. *arXiv preprint arXiv:2109.02176*, 2021.

[40] Kamran Kowsari, Kiana Jafari Meimandi, Mojtaba Heidarysafa, Sanjana Mendu, Laura Barnes, and Donald Brown. Text classification algorithms: A survey. *Information*, 10(4):150, 2019.

[41] Karim Ibrahim. Using ai-based detectors to control ai-assisted plagiarism in esl writing:“the terminator versus the machines”. *Language Testing in Asia*, 13(1):46, 2023.

[42] Irina Tolstykh, Aleksandra Tsybina, Sergey Yakubson, Aleksandr Gordeev, Vladimir Dokholyan, and Maksim Kuprashevich. Gigacheck: Detecting llm-generated content. *arXiv preprint arXiv:2410.23728*, 2024.

[43] Paul Pu Liang, Manzil Zaheer, Yuan Wang, and Amr Ahmed. Anchor & transform: Learning sparse embeddings for large vocabularies. *arXiv preprint arXiv:2003.08197*, 2020.

[44] Debadri Dutta. A review of different word embeddings for sentiment classification using deep learning. *arXiv preprint arXiv:1807.02471*, 2018.

[45] Hendri Murfi, Theresia Gowandi, Gianinna Ardaneswari, Siti Nurrohmah, et al. Bert-based combination of convolutional and recurrent neural network for indonesian sentiment analysis. *Applied Soft Computing*, 151:111112, 2024.

[46] Guoyin Wang, Chunyuan Li, Wenlin Wang, Yizhe Zhang, Dinghan Shen, Xinyuan Zhang, Ricardo Henao, and Lawrence Carin. Joint embedding of words and labels for text classification. *arXiv preprint arXiv:1805.04174*, 2018.

[47] Sarah Kreps, R Miles McCain, and Miles Brundage. All the news that's fit to fabricate: Ai-generated text as a tool of media misinformation. *Journal of experimental political science*, 9(1):104–117, 2022.

[48] Kristian Kuznetsov, Eduard Tulchinskii, Laida Kushnareva, German Magai, Serguei Barannikov, Sergey Nikolenko, and Irina Piontkovskaya. Robust ai-generated text detection by restricted embeddings. *arXiv preprint arXiv:2410.08113*, 2024.

[49] Jihad Anwar Qadir. Enhancing ai-generated text detection: A 1dcnn model withglove embeddings for nlp applications. *Available at SSRN 4952257*.

[50] David Soto-Osorio, Grigori Sidorov, Liliana Chanona-Hernández, and Blanca Cecilia López-Ramírez. Identification of scientific texts generated by large language models using machine learning. *Computers*, 13(12):346, 2024.

[51] Urwah Jawaid, Rudra Roy, Pritam Pal, Srijani Debnath, Dipankar Das, and Sivaji Bandyopadhyay. Human vs machine: An automated machine-generated text detection approach. In *Proceedings of the 21st International Conference on Natural Language Processing (ICON)*, pages 215–223, 2024.

[52] Rodrigo (thedrcat) Hernández. Augmented data for llm - detect ai generated text. https://www.kaggle.com/datasets/thedrcat/augmented-data-for-llm-detect-ai-generated-text, 2024. Accessed: 2025-04-19.

[53] BV Pranay Kumar, MD Shaheer Ahmed, and Manchala Sadanandam. Distilbert: A novel approach to detect text generated by large language models (llm). 2024.